STAT 432 Homework 07

Conner Jensen (connerj2)

Contents

Question 1: LDA **1**

(a) . . . 2

(b) . . . 2

(c) . . . 3

Question 2: QDA and marginal screening **4**

(a) . . . 5

(b) . . . 5

(c) . . . 5

Question 3: Compare LDA and QDA **6**

(a)–(b) . . . 8

(c) . . . 8

Question 4: LDA/QDA analysis skill **8**

SAheart LDA/QDA report **9**

`lda-qda-analysis` skill **9**

Question 1: LDA

```
heart <- read.csv("data/saheart.csv")
heart$famhist <- factor(heart$famhist, levels = c("Absent", "Present"))
X <- data.matrix(heart[, names(heart) != "chd"])
y <- heart$chd
n <- nrow(X)
p <- ncol(X)
K <- 2
ks <- c(0, 1)
```

(a)

Priors are $\widehat{\pi}_k = n_k/n$. Class means are column means within each `chd` group. The pooled covariance uses denominator $n - K = 460$.

```
pi_k <- sapply(ks, function(k) mean(y == k))
mu <- sapply(ks, function(k) colMeans(X[y == k, , drop = FALSE]))
colnames(mu) <- paste0("chd=", ks)
S <- matrix(0, p, p)
for (j in seq_along(ks)) {
  Xk <- scale(X[y == ks[j], , drop = FALSE], center = mu[, j], scale = FALSE)
  S <- S + crossprod(Xk)
}
Sigma <- S / (n - K)
pi_k
```

```
## [1] 0.6536797 0.3463203
```

```
round(mu, 3)
```

```
##             chd=0   chd=1
## sbp       135.460 143.738
## tobacco     2.635   5.525
## ldl         4.344   5.488
## adiposity  23.969  28.120
## famhist     1.318   1.600
## typea      52.368  54.494
## obesity    25.737  26.623
## alcohol    15.931  19.145
## age        38.854  50.294
```

```
round(Sigma, 2)
```

```
##              sbp tobacco   ldl adiposity famhist  typea obesity alcohol    age
## sbp       405.43   14.59  4.58     49.16    0.34 -15.59   18.94   64.40  95.13
## tobacco    14.59   19.24  0.76      7.54    0.02  -2.06    1.83   20.52  22.77
## ldl         4.58    0.76  4.00      6.03    0.09   0.34    2.66   -2.53   6.48
## adiposity  49.16    7.54  6.03     56.75    0.43  -5.31   22.71   16.12  60.51
## famhist     0.34    0.02  0.09      0.43    0.23   0.08    0.18    0.77   1.00
## typea     -15.59   -2.06  0.34     -5.31    0.08  95.57    2.64    7.96 -20.28
## obesity    18.94    1.83  2.66     22.71    0.18   2.64   17.62    4.69  15.70
## alcohol    64.40   20.52 -2.53     16.12    0.77   7.96    4.69  598.28  27.89
## age        95.13   22.77  6.48     60.51    1.00 -20.28   15.70   27.89 184.13
```

(b)

The LDA score at each training row is computed directly from the formula, then the larger score wins. $\mathbf{X}$ is 462×9, $\mathbf{y}$ has length 462, the means are 9×2, $\widehat{\boldsymbol{\Sigma}}$ is 9×9, and the score matrix is 462×2.

```r
invS <- solve(Sigma)
scores <- sapply(seq_along(ks), function(j) {
  as.numeric(
    X %*% invS %*% mu[, j] -
      0.5 * drop(crossprod(mu[, j], invS %*% mu[, j])) +
      log(pi_k[j])
  )
})
pred <- ks[max.col(scores, ties.method = "first")]
c(n = n, p = p, K = K)
```

```
##   n   p   K
## 462   9   2
```

```r
dim(X)
```

```
## [1] 462   9
```

```r
length(y)
```

```
## [1] 462
```

```r
dim(mu)
```

```
## [1] 9 2
```

```r
dim(Sigma)
```

```
## [1] 9 9
```

```r
dim(scores)
```

```
## [1] 462   2
```

```r
table(observed = y, predicted = pred)
```

```
##          predicted
## observed   0   1
##        0 258  44
##        1  73  87
```

```r
mean(pred != y)
```

```
## [1] 0.2532468
```

In-sample error is $117/462 \approx 0.253$. Observed `chd=0` is often correct ($258/302$); observed `chd=1` is harder ($87/160$).

(c)

```
fit <- MASS::lda(
  X,
  grouping = factor(y),
  prior = setNames(pi_k, ks)
)
pred_m <- as.integer(as.character(predict(fit)$class))
table(observed = y, predicted = pred_m)
```

```
##          predicted
## observed   0   1
##        0 258  44
##        1  73  87
```

```
all(pred == pred_m)
```

```
## [1] TRUE
```

`MASS::lda` with empirical priors gives the same confusion matrix. The hand scores and the package fit match on every training row.

Question 2: QDA and marginal screening

```
tr <- as.matrix(read.csv("data/zip-train.csv.gz", header = FALSE))
te <- as.matrix(read.csv("data/zip-test.csv.gz", header = FALSE))
ytr <- factor(tr[, 1], levels = 0:9)
yte <- factor(te[, 1], levels = 0:9)
Xtr <- tr[, -1]
Xte <- te[, -1]
v <- apply(Xtr, 2, var)
ord <- order(v, decreasing = TRUE)

fit_qda <- function(m) {
  cols <- ord[seq_len(m)]
  fit <- MASS::qda(Xtr[, cols], ytr)
  pr <- predict(fit, Xte[, cols])$class
  list(
    cm = table(observed = yte, predicted = pr),
    err = mean(pr != yte)
  )
}
q40 <- fit_qda(40)
q10 <- fit_qda(10)
q40$err
```

```
## [1] 0.1554559
```

```
q40$cm
```

```
##          predicted
## observed   0   1   2   3   4   5   6   7   8   9
##        0 319   0  24   6   2   2   1   0   5   0
##        1   0 238   5   0  11   0   7   0   3   0
##        2   8   0 168   8   8   3   0   0   3   0
##        3   5   0   4 128   0  14   0   0  15   0
##        4   0   2  13   1 150   1   0   1   7  25
##        5  12   0   3   5   1 131   0   0   6   2
##        6   0   0   8   0   1   8 148   0   5   0
##        7   1   0   6   1   6   0   0 124   2   7
##        8   1   0  10  12   3   2   0   0 137   1
##        9   0   0   1   2   7   1   1   4   9 152
```

```
q10$err
```

```
## [1] 0.3423019
```

```
q10$cm
```

```
##          predicted
## observed   0   1   2   3   4   5   6   7   8   9
##        0 298   0  11   1   1  16  28   4   0   0
##        1   0 241  10   0   3   1   1   0   2   6
##        2  19   2  95  14  14   3  22  21   5   3
##        3   5   0   6 103   3  42   0   0   7   0
##        4   1   5  14   0  95   2   4  15  17  47
##        5  15   0   6  25   4  96   4   2   8   0
##        6  41   0  15   4   0  18  78   7   5   2
##        7   4   1   6   0   6   0   1 122   0   7
##        8   1   2  11  36  12  11   4   1  74  14
##        9   0   2   9   2  20   0   0  16  10 118
```

(a)

With the 40 highest-variance pixels, QDA fits and the test misclassification rate is 0.155 (312/2007). The confusion matrix is above. Digits 0 and 1 are strongest; 4 is often called 9, and 3 is often called 5 or 8.

(b)

The method works: `qda` returns a fit and the error is well below guessing (0.9). Full 256-pixel QDA is singular because many background pixels are nearly constant within a class, so those within-class covariances are not invertible. High marginal variance drops those dead pixels. The smallest training class still has 542 images, so $n_k > 40$ after the screen.

(c)

With only 10 pixels the test error jumps to 0.342. The screen is not class-aware: the most variable pixels are not the 10 most discriminative ones, and 10 coordinates throw away too much shape. Forty pixels keep enough structure for quadratic boundaries; ten do not.

Question 3: Compare LDA and QDA

Seed 43271. Each class has 100 training and 1,000 test observations. Both methods use all 10 covariates and equal priors.

```
set.seed(43271)
rhos <- c(0, 0.2, 0.4, 0.6, 0.8, 0.9)
gen_class <- function(n, mu12, rho_sign, rho) {
  Sig <- matrix(c(1, rho_sign * rho, rho_sign * rho, 1), 2, 2)
  z <- matrix(rnorm(n * 2), n, 2) %*% chol(Sig)
  cbind(z[, 1] + mu12[1], z[, 2] + mu12[2],
        matrix(rnorm(n * 8), n, 8))
}
gen_xy <- function(n_per, rho) {
  X <- rbind(
    gen_class(n_per, c(-1, 0), 1, rho),
    gen_class(n_per, c(1, 0), -1, rho)
  )
  list(X = X, y = factor(rep(1:2, each = n_per)))
}

one <- sapply(rhos, function(rho) {
  trn <- gen_xy(100, rho)
  tes <- gen_xy(1000, rho)
  fl <- MASS::lda(trn$X, trn$y, prior = c(0.5, 0.5))
  fq <- MASS::qda(trn$X, trn$y, prior = c(0.5, 0.5))
  c(
    lda = mean(predict(fl, tes$X)$class != tes$y),
    qda = mean(predict(fq, tes$X)$class != tes$y)
  )
})
rbind(rho = rhos, round(one, 4))
```

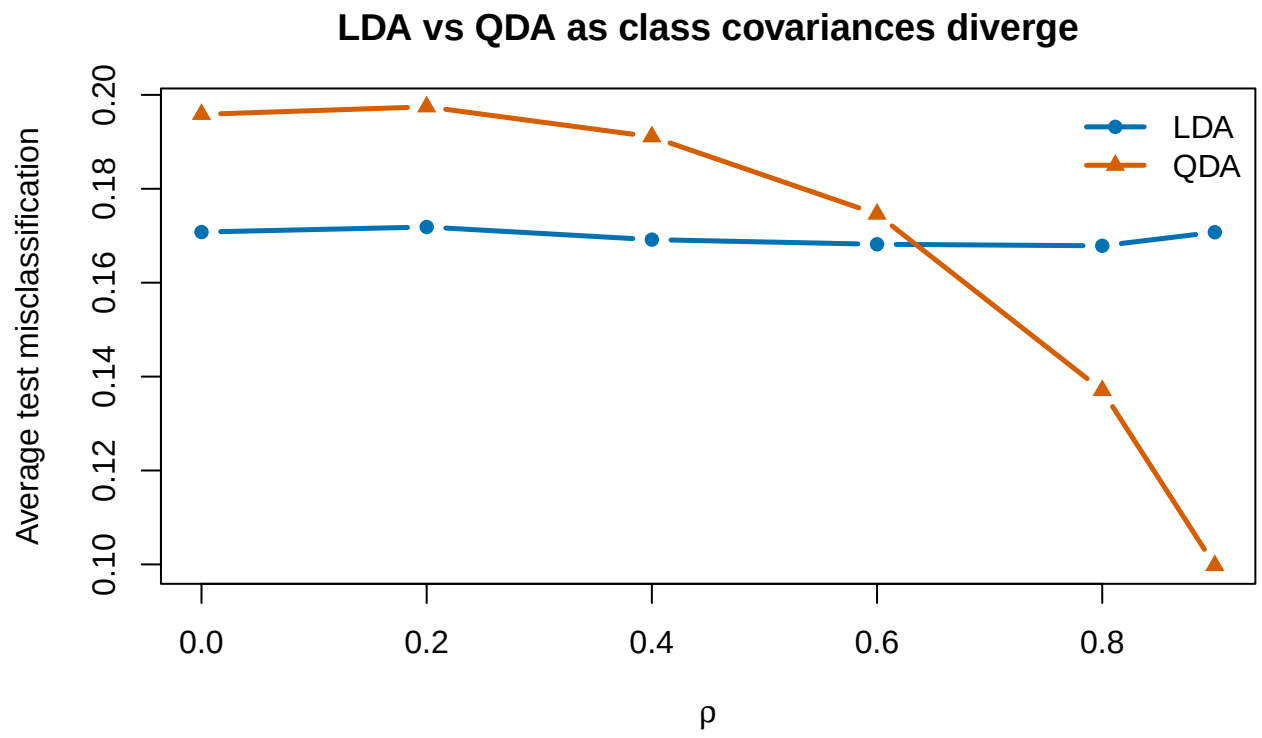
```
##       [,1]  [,2]  [,3]  [,4]   [,5]  [,6]
## rho 0.0000 0.200 0.400 0.600 0.8000 0.900
## lda 0.1865 0.168 0.170 0.166 0.1540 0.165
## qda 0.2140 0.195 0.193 0.179 0.1225 0.097
```

```
set.seed(43271)
nrep <- 50
avg <- matrix(0, 2, length(rhos),
              dimnames = list(c("lda", "qda"), NULL))
for (r in seq_len(nrep)) {
  for (j in seq_along(rhos)) {
    trn <- gen_xy(100, rhos[j])
    tes <- gen_xy(1000, rhos[j])
    fl <- MASS::lda(trn$X, trn$y, prior = c(0.5, 0.5))
    fq <- MASS::qda(trn$X, trn$y, prior = c(0.5, 0.5))
    avg["lda", j] <- avg["lda", j] +
      mean(predict(fl, tes$X)$class != tes$y)
    avg["qda", j] <- avg["qda", j] +
      mean(predict(fq, tes$X)$class != tes$y)
  }
```

```
}
avg <- avg / nrep
data.frame(rho = rhos, lda = avg["lda", ], qda = avg["qda", ])
```

```
##   rho     lda     qda
## 1 0.0 0.17078 0.19585
## 2 0.2 0.17186 0.19746
## 3 0.4 0.16917 0.19108
## 4 0.6 0.16818 0.17461
## 5 0.8 0.16786 0.13705
## 6 0.9 0.17075 0.09977
```

```
par(mar = c(4.4, 4.4, 3.2, 1.1))
plot(rhos, avg["lda", ], type = "b", lwd = 2.5, pch = 16,
     col = "#0072B2", ylim = range(avg),
     xlab = expression(rho), ylab = "Average test misclassification",
     main = "LDA vs QDA as class covariances diverge")
lines(rhos, avg["qda", ], type = "b", lwd = 2.5, pch = 17,
      col = "#D55E00")
legend("topright", bty = "n",
       legend = c("LDA", "QDA"),
       col = c("#0072B2", "#D55E00"), lwd = 2.5, pch = c(16, 17))
```



(a)–(b)

On a single draw, LDA is better at small ρ and QDA is better at $\rho = 0.8$ and 0.9. Averaged over 50 repeats, LDA stays near 0.17 for every ρ. QDA starts worse (about 0.196 at $\rho = 0$) and then drops to about 0.100 at $\rho = 0.9$.

(c)

When $\rho = 0$ the two classes share the same covariance, so LDA is correctly specified. QDA estimates two 10×10 covariance matrices from 100 observations each, which adds variance and raises error. As ρ grows, the class covariances truly differ (ρ versus $-\rho$). That is bias for the pooled LDA matrix and signal for QDA, so QDA overtakes LDA.

The eight noise covariates do not change the Bayes rule, but both methods still estimate their covariances in those directions. QDA pays a larger variance cost because it fits a separate matrix in each class. That is why QDA loses when ρ is small, even though its model contains LDA as a special case.

Question 4: LDA/QDA analysis skill

I wrote a short skill `lda-qda-analysis` (1876 bytes) and kept it in `homework/week-07/lda-qda-analysis/SKILL.md` instead of `.cursor/skills`, with `disable-model-invocation: true`, so it only runs when I ask for it.

I applied it with this prompt:

> Use the lda-qda-analysis skill on the SAheart data from Question 1. Dataset: `data/saheart.csv`. Response: `chd`. Predictors: the nine other variables, with `famhist` coded Absent = 1 and Present = 2. Seek the best predictive performance within the LDA/QDA methods allowed by the skill.

The one-page report is next. After checking the training-fold CV results, I left the skill as is. The model was chosen from those folds only.

SAheart LDA/QDA report

Seed 43272. I held out a stratified 30% test set (138 observations) and used 10-fold CV on the remaining 324. The same folds are used for every candidate. Any centering, scaling, or PCA is estimated on the training part of a fold only.

I compared ordinary LDA, ordinary QDA, a 50-50 blend of their class-1 posteriors, and LDA or QDA on the first $d = 2, \ldots, 8$ principal components. QDA on 8 PCs had the lowest mean CV error (0.259). Ordinary LDA was close (0.272). After refitting on all training rows, the selected model has test error 0.290.

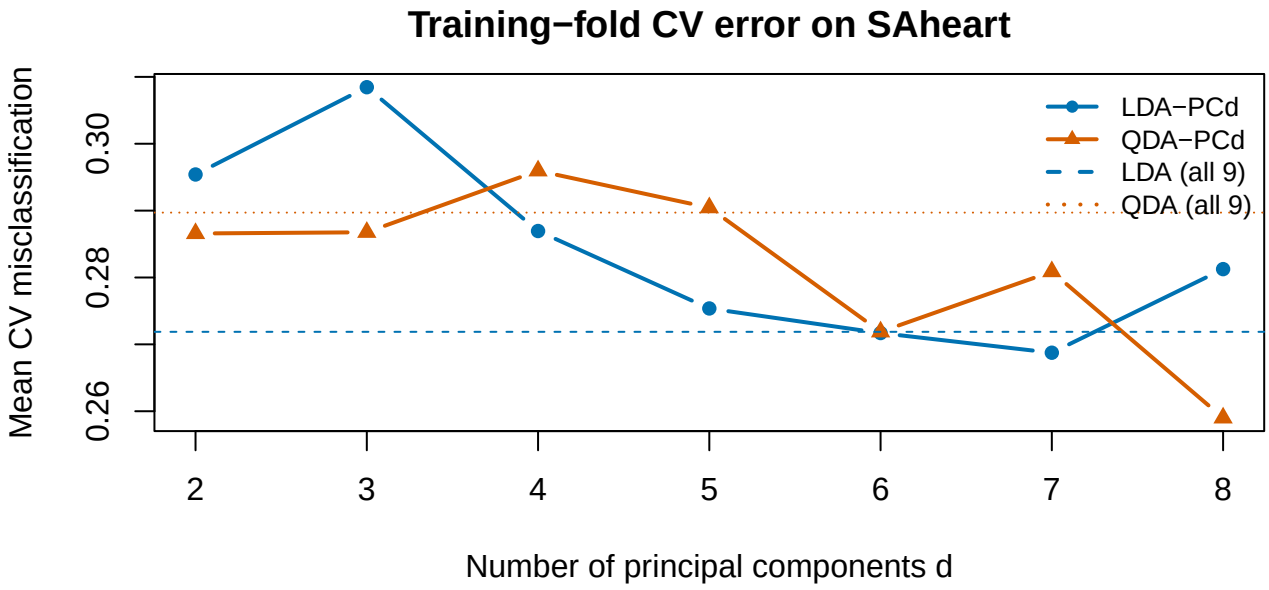


Table 1: Mean CV error and test error.

model	cv_error	test_error
LDA	0.272	0.275
QDA	0.290	0.297
Blend	0.281	0.290
LDA-PC7	0.269	0.275
QDA-PC8	0.259	0.290

Ordinary QDA on all nine variables is a bit worse than LDA, so the extra covariance parameters do not help here. QDA on 8 PCs wins CV, but only by a little, and it does not beat LDA on the unused test set (0.290 vs 0.275). The sample is not large, `famhist` is just a 1/2 numeric code, and I only compared LDA/QDA-type rules. A different split could change which model wins CV.

`lda-qda-analysis` skill

```
---
name: lda-qda-analysis
```

```
description: >
  Run an LDA/QDA analysis and write a one-page report.
  Use only when the student explicitly asks to use
  lda-qda-analysis on a supplied dataset.
disable-model-invocation: true
---

# LDA/QDA analysis

Use this skill only after an explicit request. The student
supplies the dataset, the response, and the predictors.
Do not assume variable names or results.

## Allowed models

Compare ordinary LDA and ordinary QDA. Optional extensions,
still based only on LDA or QDA:

- LDA or QDA on a PCA embedding of the predictors
- a blend of LDA and QDA class probabilities

Do not add other classifiers.

## Protocol

1. Set and report a random seed.
2. Reserve a test set before any model comparison.
   Do not use test labels to choose a model or setting.
3. On the training data, compare candidates with the same
   cross-validation folds. Estimate centering, scaling, or
   PCA using only the training portion of each fold, then
   apply that map to the held-out fold.
4. Select the candidate with the lowest mean validation
   misclassification rate. If two are tied, prefer the
   simpler model (ordinary LDA, then ordinary QDA, then
   fewer principal components, then a blend).
5. Refit the selected model and its preprocessing on all
   training observations. Evaluate that fitted classifier
   once on the reserved test set.

## One-page report

Write at most one printed page containing:

- a short description of the candidate models
- the cross-validation scheme and the mean validation
  misclassification rates
- the selected model and its estimated prediction error
  (validation rate, then the unused test rate)
- one figure (validation error across candidates)
- one table (validation and test misclassification rates)
- a brief statement of findings and limitations
```

```
Do not exceed one page. Do not tune on the test set.
```